Отчет по изменению
Индивидуального Домашнего Задания №1
Вариант 35

Сделано
Жарковой Татьяной Владимировной
студенткой 2 курса Программной инженерии
группы БПИ-253

Москва 2026
Высшая школа экономики

Оглавление

Аннотация 4

Изменения в изначальной версии 5

Изменение структуры `EventType` 5

Добавление структуры `StopReason` 5

Обновление структуры `District` 5

Обновление структуры `Simulation` 6

Изменения в config.h 6

Изменения в event_queue.h 6

Изменения в simulation.h 7

Изменения в utils_extra.h 7

Новый файл generator.h 7

Изменение config.c 8

Изменение `int set_defaults(Simulation *sim)` 8

Новая функция `int parse_int_strict(const char *text, int *value)` 8

Изменение `static int parse_settings_line(Simulation *sim, const char *line)` 9

Изменение в `int load_config(Simulation *sim, const char *path)` 9

Изменение в `int validate_simulation(const Simulation *sim)` 9

Добавление новых функций в `event_queue.c` 9

Изменение в `dispatch.c` 11

Изменения в `event_handlers.c` 12

Добавлена функция `static int waiting_calls(const Simulation *sim)` 12

Изменения в `handle_call_arrival` 12

Переработана `handle_escalation` 13

Добавлена функция `handle_wait_timeout` 14

Изменения в `simulation.c` 15

Изменение константы 15

Флаги остановки 15

Изменение `on_signal()` 15

Добавлена функция `process_signal_requests()` 16

Добавлена флаг-функция `stop_is_requested()` 16

Изменение `seed_events()` 16

Добавлена функция `check_invariants()` 17

Расширение функции `mark_active_calls_unserved()` 18

Добавлена функция `report_not_arrived()` 18

Переработана функция `run_simulation()` 18

Изменения `install_signal()` 19

Новый файл `generator.c` 19

Функция `random_between()` 19

Функция `find_free_call_slot()` 20

Функция `create_random_call()` 20

Функция `schedule_next_random_call` 21

Изменения в `utils.c` 22

Добавлена функция `write_all()` 22
Добавлена функция `fd_printf` 22
Изменение `log_event()` 24
Изменение в `output.c` 24
Изменение `main.c` 25
Добавление констант 25
Изменение `parse_arguments()` 25
Добавлена проверка лишних аргументов 25
Изменение функции `open_log_file()` 26

Аннотация

В данном отчете представлена работа над ошибками индивидуального домашнего задания, вариант 35. Ниже будут описаны важные изменения оригинальной программы.

Изменения в изначальной версии

Изменение структуры `EventType`

Был добавлен тип события - `EV_WAIT_TIMEOUT` в структуру `EventType`. Он отвечает за обработку события отмены вызова по времени. Без отдельного события программа может обнаружить превышение времени ожидания только тогда, когда произойдёт какое-нибудь другое событие. Поэтому эта ошибка была устранена. Теперь программа не зависит от того, когда произойдет следующее событие, чтобы обнаружить просроченный вызов.

Добавление структуры `StopReason`

Была добавлена структура `StopReason`. Причины остановки программы хранятся явно.

```
typedef enum
{
    STOP_ALL_PROCESSED = 0,
    STOP_SHIFT_LIMIT,
    STOP_INTERRUPTED,
    STOP_INTAKE_CLOSED
} StopReason;
```

- `STOP_ALL_PROCESSED` - все вызовы обработаны,
- `STOP_SHIFT_LIMIT` - смена закончена,
- `STOP_INTERRUPTED` - пользователь остановил программу,
- `STOP_INTAKE_CLOSED` - Получен SIGQUIT, приём новых вызовов закрыт

Обновление структуры `District`

Добавлено поле `int travel_count` - сколько всего возможностей передвижения команды. Теперь количество заданных времён должно совпадать с количеством районов.

Обновление структуры `Simulation`

- int received - количество вызовов, которые действительно поступили в систему;
- int invariant_errors - счётчик нарушений инвариантов;
- StopReason stop_reason - причина остановки программы;
- int random_calls - флаг включения генератора случайный чисел;
- unsigned int seed - начальное значение генератора случайных чисел;
- unsigned int rng - текущее состояние генератора;
- int random_interval - интервал, определяющий частоту поступления случайных вызовов;
- int next_call_id - идентификатор следующего генерируемого вызова;
- int intake_closed - Показывает, закрыт ли приём новых вызовов. Используется при SIGQUIT. Он должен просто прекратить поступление новых вызовов, но не обрывать саму симуляцию;
- int log_fd - файловый дескриптор POSIX;

Изменения в config.h

Добавление функции `int parse_int_strict(const char *text, int *value)`. Дополнительная валидация перевода строки в число.

Изменения в event_queue.h

Добавление функций:

- `int remove_call_deadlines(Simulation *sim, int call_id)` - удаление событий , связанных с ожидании бригады (escalation, cancel, wait_timeout), из очереди.

- `int remove_call_arrivals(Simulation *sim)` - удаление событий, связанных с будущими поступлениями (arrival), из очереди.

Изменения в simulation.h

- Было удалено объясление `travel_time()`. Оно осталось только в логике диспетчеризации;

- Был создан `GENERATED_CALL` - специальный идентификатор события поступления случайного вызова;

- Была добавлена `int check_invariants(Simulation *sim)` - функция для проверки согласованности модели;

- Была добавлена `int stop_is_requested(void)`- функция-флаг для проверки получения сигнала о прекращении работы симуляции.

Изменения в utils_extra.h

- Была добавлена `OUT_BUF_LEN` - длина буфера;

- Была добавлена `int write_all(int fd, const char *buf, size_t len)` - функция для **полной записи** переданного буфера в файловый дескриптор;

- Была добавлена `int fd_printf(int fd, const char *format, ...) __attribute__((format(printf, 2, 3)))` - функция для форматированной записи в дескриптор.

Новый файл generator.h

Для генерации новых вызовов.

- `int create_random_call(Simulation *sim)` - функция создает новый вызов и записывает его в массив;

- `int schedule_next_random_call(Simulation *sim)` - функция добавляет в очередь событие будущего поступления случайного вызова.

Изменение config.c

Изменение `int set_defaults(Simulation *sim)`

Добавление:

- sim->log_fd = -1 - файл журнала ещё не открыт;
- sim->stop_reason = STOP_ALL_PROCESSED - начальная причина остановки;
- sim->random_calls = 0 - генератор выключен по умолчанию;
- sim->seed = 1 - начальное значение генератора;
- sim->random_interval = 10 - интервал генерации по умолчанию;
- sim->next_call_id = 1 - id следующего звонка;

Новая функция `int parse_int_strict(const char *text, int *value)`

```c
/**
 * @brief Converts a string to int and checks that the whole string is a number
 * @param text Text to convert
 * @param value Where to store the result
 * @return 1 on success, 0 if the text is not an integer
 */
int parse_int_strict(const char *text, int *value)
{
    SOFT_ASSERT(text, "text is NULL", 0);
    SOFT_ASSERT(value, "value pointer is NULL", 0);

    char *end = NULL;
    errno = 0;
    long number = strtol(text, &end, 10);

    if (end == text || *end != '\0' || errno == ERANGE || number < INT_MIN || number > INT_MAX)
        return 0;

    *value = (int)number;
    return 1;
}
```

Изменение `static int parse_settings_line(Simulation *sim, const char *line)`

```
int number = 0;
if (!parse_int_strict(val, &number))
{
    fprintf(stderr, "Config error: value of '%s' must be an integer, got '%s'\n", key, val);
    return 0;
}
```

- формат sscanf() стал учитывать пробелы вокруг разделителя =;
- при ошибке теперь выводится сообщение;
- функция возвращает 0, а не 1.

Изменение в `int load_config(Simulation *sim, const char *path)`

```
if (sim->district_count == 0 || sim->team_count == 0)
{
    fprintf(stderr, "Config error: %s must contain [DISTRICTS] and [TEAMS] sections\n", path);
    return 0;
}
```

- проверка записана явно;
- выводится понятная ошибка;
- указывается путь к конфигурации.

Изменение в `int validate_simulation(const Simulation *sim)`

```
SOFT_ASSERT(sim->shift_limit >= 0, "shift limit must be non-negative (0 = unlimited)", 0);
SOFT_ASSERT(sim->random_calls == 0 || sim->random_calls == 1, "random_calls must be 0 or 1", 0);
SOFT_ASSERT(sim->random_interval > 0, "random_interval must be positive", 0);
SOFT_ASSERT(sim->call_count > 0 || sim->random_calls, "no calls configured and the random generator is off", 0);
```

Изменение проверок и добавление новых

Добавление новых функций в `event_queue.c`

```c
/**
 * @brief Restores the heap property for the subtree rooted at idx
 * @param sim Pointer to the simulation structure
 * @param idx Index of the root of the subtree
 */
static void sift_down(Simulation *sim, int idx)
{
    for (;;)
    {
        int left = idx * 2 + 1;
        int right = left + 1;
        int best = idx;

        if (left < sim->event_count && event_less(&sim->events[left], &sim->events[best]))
            best = left;

        if (right < sim->event_count && event_less(&sim->events[right], &sim->events[best]))
            best = right;

        if (best == idx)
            return;

        Event temp = sim->events[idx];
        sim->events[idx] = sim->events[best];
        sim->events[best] = temp;

        idx = best;
    }
}
```

Создана для восстановлении структуры бинарной кучи после удаления события из очереди.

```c
/**
 * @brief Removes the personal deadlines (escalation, cancellation, max_wait)
 * of a call from the queue
 * @param call_id Id of the call
 * @return Number of removed events
 */
int remove_call_deadlines(Simulation *sim, int call_id)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);

    int kept = 0;
    int removed = 0;

    for (int idx = 0; idx < sim->event_count; idx++)
    {
        const Event *event = &sim->events[idx];
        int deadline = event->type == EV_ESCALATION || event->type == EV_CANCEL || event->type == EV_WAIT_TIMEOUT;

        if (deadline && event->id == call_id)
        {
            removed++;
            continue;
        }

        sim->events[kept++] = *event;
    }

    sim->event_count = kept;

    // rebuild the binary heap (Floyd's algorithm)
    for (int idx = sim->event_count / 2 - 1; idx >= 0; idx--)
        sift_down(sim, idx);

    return removed;
}
```

Создана для удаления из очереди все отложенные события, связанные с:

- повышением приоритета (EV_ESCALATION);
- отменой вызова (EV_CANCEL);
- истечением максимального времени ожидания (EV_WAIT_TIMEOUT).

```c
/**
 * @brief Removes all future call arrivals from the queue (intake is closed)
 * @param sim Pointer to the simulation structure
 * @return Number of removed arrivals
 */
int remove_call_arrivals(Simulation *sim)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);

    int kept = 0;
    int removed = 0;

    for (int idx = 0; idx < sim->event_count; idx++)
    {
        if (sim->events[idx].type == EV_CALL_ARRIVAL)
        {
            removed++;
            continue;
        }

        sim->events[kept++] = sim->events[idx];
    }

    sim->event_count = kept;

    for (int idx = sim->event_count / 2 - 1; idx >= 0; idx--)
        sift_down(sim, idx);

    return removed;
}
```

Создана для удаления всех событий поступления вызовов (EV_CALL_ARRIVAL), независимо от их идентификатора. Это сделано для того, чтобы далить запланированные события будущих поступлений, например, при получения SIGQUIT.

Изменение в `dispatch.c`

Добавлена remove_call_deadlines(sim, call_->id).Теперь при назначении бригады из очереди удаляются связанные с этим вызовом события:

- EV_ESCALATION;

- EV_CANCEL;
- EV_WAIT_TIMEOUT.

Вызов больше не находится в очереди ожидания, поэтому ему не нужно повышать приоритет или отменяться по таймеру ожидания.

Изменения в `event_handlers.c`

Добавлена функция `static int waiting_calls(const Simulation *sim)`

```c
/**
 * @brief Counts calls that are waiting for a team
 * @param sim Pointer to the simulation structure
 * @return Number of calls in state CALL_WAITING
 */
static int waiting_calls(const Simulation *sim)
{
    int count = 0;

    for (int idx = 0; idx < sim->call_count; idx++)
        if (sim->calls[idx].state == CALL_WAITING)
            count++;

    return count;
}
```

Функция принимает указатель на симуляцию и возвращает количество вызовов, которые ожидают назначения бригады.

Изменения в `handle_call_arrival`

```c
int idx = -1;

if (event->id == GENERATED_CALL)
{
    // random generator: create the call now and plan the next one
    if (sim->intake_closed)
        return 1;

    idx = create_random_call(sim);
    schedule_next_random_call(sim);

    if (idx < 0)
        return 1; // the call table is full, the call was rejected and reported
} else
    idx = find_call_index_by_id(sim, event->id);
```

Генерация звонков

```c
// Schedule the personal deadlines of the call
if (sim->escalation_enabled && call->escalation_after > 0 && call->priority > 1)
    schedule_event_checked(sim, call->arrival + call->escalation_after, EV_ESCALATION, call->id);

if (call->cancel_after > 0)
    schedule_event_checked(sim, call->arrival + call->cancel_after, EV_CANCEL, call->id);

// the call is fixed as unserved exactly when max_wait expires
schedule_event_checked(sim, call->arrival + call->max_wait, EV_WAIT_TIMEOUT, call->id);

try_dispatch(sim);

if (call->state == CALL_WAITING)
    log_event(sim, "SEARCH: no free %s team can reach %s; CALL #%03d is queued (waiting calls: %d)",
              spec_name(call->spec), sim->districts[call->district].name, call->id, waiting_calls(sim));
```

Добавлена проверка приоритета, добавлено событие `EV_WAIT_TIMEOUT`.

```c
if (call->state == CALL_WAITING)
    log_event(sim, "SEARCH: no free %s team can reach %s; CALL #%03d is queued (waiting calls: %d)",
              spec_name(call->spec), sim->districts[call->district].name, call->id, waiting_calls(sim));

sleep_visual(sim);
```

Теперь после попытки назначить бригаду проверяется состояние вызова.

Переработана `handle_escalation`

```c
static int handle_escalation(Simulation *sim, const Event *event)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);
    SOFT_ASSERT(event, "event is NULL", 0);

    int idx = find_call_index_by_id(sim, event->id);
    if (idx < 0)
        return 1; // the call slot was reused by the random generator

    Call *call = &sim->calls[idx];
    if (call->state != CALL_WAITING || call->priority <= 1)
        return 1;

    int old_priority = call->priority;
    call->priority--;
    sim->escalations++;

    log_event(sim, "CALL #%03d PRIORITY ESCALATED %d -> %d after %d min of waiting", call->id, old_priority,
              call->priority, sim->now - call->arrival);

    // next escalation step, if the call can still become more urgent
    if (call->priority > 1)
        schedule_event_checked(sim, sim->now + call->escalation_after, EV_ESCALATION, call->id);

    try_dispatch(sim);

    sleep_visual(sim);

    return 1;
}
```

Изменена проверка состояния, сохраняется старый приоритет, повышение приоритета может происходить несколько раз, после каждого повышения приоритета вызываем `try_dispatch`.

Добавлена функция `handle_wait_timeout`

```c
static int handle_wait_timeout(Simulation *sim, const Event *event)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);
    SOFT_ASSERT(event, "event is NULL", 0);

    int call_idx = find_call_index_by_id(sim, event->id);
    if (call_idx < 0)
        return 1; // the call slot was reused by the random generator

    Call *call = &sim->calls[call_idx];

    if (call->state != CALL_WAITING)
        return 1;

    call->state = CALL_UNSERVED;
    remove_call_deadlines(sim, call->id);
    sim->unserved++;

    log_event(sim, "CALL #%03d UNSERVED: maximum waiting time %d min exceeded", call->id, call->max_wait);

    return 1;
}
```

Это обработчик события `EV_WAIT_TIMEOUT`.

Изменения в `simulation.c`

Изменение константы

`SNAPSHOT_EVERY_EVENTS 4` - константа отвечает за количество обработанных событий, после которого нужно вывести снимок состояния.

Флаги остановки

Появилось 3 флага остановки:

- `stop_requested` - срочно остановить симуляцию;
- `close_requested` - прекратить приём новых вызовов, но обработать уже принятые;
- `snapshot_requested` - вывести текущее состояние симуляции.

Изменение `on_signal()`

```c
/**
 * @brief Signal handler: only sets flags, the main loop reacts to them
 * @param number Signal number
 */
static void on_signal(int number)
{
    if (number == SIGINT || number == SIGTERM)
        stop_requested = 1;
    else if (number == SIGQUIT)
        close_requested = 1;
    else if (number == SIGUSR1)
        snapshot_requested = 1;
}
```

Теперь один обработчик отвечает за четыре сигнала:

- SIGINT - при нажатии Ctrl+C;
- SIGTERM - стандартный сигнал завершения процесса;
- SIGQUIT - при нажатии Ctrl+;
- SIGUSR1 - пользовательский сигнал.

Добавлена функция `process_signal_requests()`

```c
/**
 * @brief Reacts to SIGQUIT and SIGUSR1 between two events
 * @param sim Pointer to the simulation structure
 */
static void process_signal_requests(Simulation *sim)
{
    if (close_requested && !sim->intake_closed)
    {
        sim->intake_closed = 1;
        sim->random_calls = 0; // the generator is stopped as well
        int removed = remove_call_arrivals(sim);
        log_event(sim, "INTAKE CLOSED BY USER: %d future call(s) will not be accepted, active calls are finished", removed);
    }

    if (snapshot_requested)
    {
        snapshot_requested = 0;
        show_snapshot(sim);
    }
}
```

Эта функция обработки сигналов SIGQUIT и SIGUSR1.

Добавлена флаг-функция `stop_is_requested()`

```c
/**
 * @brief Returns 1 if the user requested to stop the simulation
 */
int stop_is_requested(void)
{
    return stop_requested != 0;
}
```

Это флаг-функция остановки симуляции

Изменение `seed_events()`

```c
if (sim->random_calls)
{
    // generated ids continue after the largest configured id
    sim->next_call_id = 1;
    for (int idx = 0; idx < sim->call_count; idx++)
        if (sim->calls[idx].id >= sim->next_call_id)
            sim->next_call_id = sim->calls[idx].id + 1;

    sim->rng = sim->seed;
    int first = (int)(rand_r(&sim->rng) % (unsigned)sim->random_interval);

    SOFT_ASSERT(schedule_event_checked(sim, first, EV_CALL_ARRIVAL, GENERATED_CALL),
                "cannot schedule generated call", 0);
}
```

Добавлена генерация случайных вызовов

Добавлена функция `check_invariants()`

(Слишком большое фото, не поместилось, простите)

Функция проверяет, что внутреннее состояние симуляции не противоречит своим законам, инвариантам.

Основные инварианты:

1. Корректность количества объектов: количество районов, бригад и вызовов не должно выходить за допустимые границы массивов;
2. Уникальность идентификаторов: у районов, бригад и вызовов не должно быть повторяющихся ID;
3. Корректность районов: каждый район должен иметь допустимые параметры, а матрица времени перемещения — быть полной, содержать неотрицательные значения и нули на диагонали;
4. Корректность бригад: каждая бригада должна иметь допустимую специализацию и ссылаться на существующий район;
5. Корректность назначений: если бригада занята вызовом, этот вызов должен существовать и ссылаться на ту же бригаду. Свободная бригада должна иметь `call_id == NO_CALL`;
6. Корректность вызовов: каждый вызов должен ссылаться на существующий район, иметь допустимый приоритет и корректное состояние;
7. Согласованность связей: назначенная вызову бригада должна существовать, быть занята и иметь соответствующий `call_id`;
8. Корректность состояний: завершенные, отменённые и необслуженные вызовы не должны оставаться назначенными бригадам;
9. Корректность статистики: счетчики событий, поступивших вызовов и других операций не должны противоречить состоянию симуляции.

Расширение функции `mark_active_calls_unserved()`

```c
static int mark_active_calls_unserved(Simulation *sim, const char *reason)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);
    SOFT_ASSERT(reason, "reason is NULL", 0);

    for (int idx = 0; idx < sim->call_count; idx++)
    {
        Call *call = &sim->calls[idx];

        if (call->state != CALL_WAITING && call->state != CALL_EN_ROUTE && call->state != CALL_IN_SERVICE)
            continue;

        if (call->assigned_team != NO_TEAM)
            log_event(sim, "CALL #%03d UNSERVED: %s (state %s, team B%d)", call->id, reason, call_state_name(call->state), call->assigned_team);
        else
            log_event(sim, "CALL #%03d UNSERVED: %s (state %s)", call->id, reason, call_state_name(call->state));

        call->state = CALL_UNSERVED;
        sim->unserved++;
    }

    return 1;
}
```

Добавлена причина, почему вызовы не обслужены, проверяются и логгируются активные вызовы.

Добавлена функция `report_not_arrived()`

```c
/**
 * @brief Prints the calls that never arrived (their arrival is after the stop)
 * @param sim Pointer to the simulation structure
 */
static void report_not_arrived(Simulation *sim)
{
    int count = 0;

    for (int idx = 0; idx < sim->call_count; idx++)
        if (sim->calls[idx].state == CALL_PENDING)
            count++;

    if (count > 0)
        log_event(sim, "%d call(s) were scheduled after the end of the simulation and never arrived", count);
}
```

Эта функция считает вызовы, которые были запланированы, но так и не поступили.

Переработана функция `run_simulation()`

(фотография слишком большая, простите( )

- Добавлена проверка результата `seed_events()`
- Инициализируется причина завершения с помощью `SropReason`

- Изменено условие основного цикла (добавлена возможность обработать запросы SIGQUIT и SIGUSR1, даже если очередь событий уже пуста)
- В начале каждой итерации обрабатываются сигналы
- Добавлена проверка пустой очереди
- Добавлена обработка немедленной остановки
- Изменена проверка окончания смены
- Событие извлекается только после всех проверок
- Добавлена проверка инвариантов после события
- Снимки состояния выводятся по количеству событий
- Добавлена обработка причины завершения симуляции
- Добавлен отчёт о вызовах, которые не поступили

Изменения `install_signal()`

```c
/**
 * @brief Installs handlers for SIGINT (Ctrl+C), SIGTERM, SIGQUIT (Ctrl+\\) and SIGUSR1
 * @return 1 on success, 0 on failure
 */
int install_signal(void)
{
    struct sigaction action;
    memset(&action, 0, sizeof(action));
    action.sa_handler = on_signal;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    SOFT_ASSERT(sigaction(SIGINT, &action, NULL) == 0, "cannot install SIGINT handler", 0);
    SOFT_ASSERT(sigaction(SIGTERM, &action, NULL) == 0, "cannot install SIGTERM handler", 0);
    SOFT_ASSERT(sigaction(SIGQUIT, &action, NULL) == 0, "cannot install SIGQUIT handler", 0);
    SOFT_ASSERT(sigaction(SIGUSR1, &action, NULL) == 0, "cannot install SIGUSR1 handler", 0);

    return 1;
}
```

Она регистрирует обработчик для всех 4 сигналов: SIGINT, SIGTERM, SIGQUIT, SIGUSR1.

Новый файл `generator.c`

Функция `random_between()`

```c
/**
 * @brief Returns a pseudo-random number
 * @param sim Pointer to the simulation structure
 * @param low Lower bound
 * @param high Upper bound
 * @return Random number
 */
static int random_between(Simulation *sim, int low, int high)
{
    if (high <= low)
        return low;

    return low + (int)(rand_r(&sim->rng) % (unsigned)(high - low + 1));
}
```

Эта функция генерирует псевдослучайное число от low до high.

Функция `find_free_call_slot()`

```c
/**
 * @brief Finds a slot for a new call
 * @param sim Pointer to the simulation structure
 * @return Index of the slot or -1 if all calls are still active
 */
static int find_free_call_slot(Simulation *sim)
{
    if (sim->call_count < MAX_CALLS)
        return sim->call_count++;

    for (int idx = 0; idx < sim->call_count; idx++)
    {
        CallState state = sim->calls[idx].state;
        if (state == CALL_COMPLETED || state == CALL_CANCELLED || state == CALL_UNSERVED)
            return idx;
    }

    return -1;
}
```

Эта функция ищет свободное место в массиве для записи нового вызова.

Функция `create_random_call()`

(слишком большое фото( )

Эта функция генерации случайного вызова. Она выбирает район, приоритет и необходимые параметры, присваивает вызову уникальный идентификатор и добавляет его в симуляцию.

Функция `schedule_next_random_call`

```c
/**
 * @brief Plans the arrival of the next random call
 * @param sim Pointer to the simulation structure
 * @return 1 on success, 0 on failure
 */
int schedule_next_random_call(Simulation *sim)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);

    int gap = random_between(sim, 1, 2 * sim->random_interval - 1);

    return schedule_event_checked(sim, sim->now + gap, EV_CALL_ARRIVAL, GENERATED_CALL);
}
```

Эта функция планирует следующее поступление случайного вызова.

Изменения в `utils.c`

Добавлена функция `write_all()`

```c
/**
 * @brief Writes the whole buffer to a file descriptor
 * @param fd File descriptor
 * @param buf Data to write
 * @param len Number of bytes
 * @return 1 on success, 0 on failure
 */
int write_all(int fd, const char *buf, size_t len)
{
    SOFT_ASSERT(buf, "buffer is NULL", 0);

    while (len > 0)
    {
        ssize_t written = write(fd, buf, len);

        if (written < 0)
        {
            if (errno == EINTR) // interrupted by SIGINT before anything was written
                continue;
            return 0;
        }

        buf += written;
        len -= (size_t)written;
    }

    return 1;
}
```

Это обёртка над системным вызовом write(), которая пытается записать весь переданный буфер. Это делается для того, чтобы весь буфер был записан. Потому что POSIX не гарантирует, что за один вызов будут записаны все переданные байты

Добавлена функция `fd_printf`

```c
/**
 * @brief printf-like output to a file descriptor (vsnprintf + write(2))
 * @param fd File descriptor (STDOUT_FILENO, log descriptor, ...)
 * @param fmt Format string
 * @return 1 on success, 0 on failure
 */
int fd_printf(int fd, const char *fmt, ...)
{
    SOFT_ASSERT(fmt, "format is NULL", 0);

    if (fd < 0)
        return 1; // nothing to write to (log is disabled)

    char buffer[OUT_BUF_LEN];

    va_list ap;
    va_start(ap, fmt);
    int len = vsnprintf(buffer, sizeof(buffer), fmt, ap);
    va_end(ap);

    SOFT_ASSERT(len >= 0, "formatting error", 0);

    if ((size_t)len >= sizeof(buffer)) // the line was truncated
        len = (int)sizeof(buffer) - 1;

    return write_all(fd, buffer, (size_t)len);
}
```

Эта функция - аналог `printf()` для файлового дескриптора.

Изменение `log_event()`

```c
int log_event(Simulation *sim, const char *fmt, ...)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);
    SOFT_ASSERT(fmt, "log format is NULL", 0);

    char message[OUT_BUF_LEN];

    va_list ap;         // Variable argument list
    va_start(ap, fmt); // Initialize the variable argument list
    vsnprintf(message, sizeof(message), fmt, ap);
    va_end(ap);

    // model time is in minutes: hours = now / 60, minutes = now % 60
    char line[OUT_BUF_LEN + 32];
    int len = snprintf(line, sizeof(line), "[%02d:%02d] %s\n", sim->now / 60, sim->now % 60, message);

    if (len < 0)
        return 0;

    if ((size_t)len >= sizeof(line))
        len = (int)sizeof(line) - 1;

    write_all(STDOUT_FILENO, line, (size_t)len);

    if (sim->log_fd >= 0)
        write_all(sim->log_fd, line, (size_t)len);

    return 1;
}
```

Добавлен промежуточный буфер сообщения, изменён формат времени, добавлен буфер полной строки

Изменение в `output.c`

Появились:

- стиль оформления;
- вывод через файловые дескрипторы;
- отображение причины завершения;
- поддержка неограниченной смены;
- отображение параметров генератора;
- более подробный снимок состояния;
- отдельная функция записи итоговой статистики в лог;
- новый счётчик нарушений инвариантов.

Изменение `main.c`

Добавление констант

- ARGS_ERROR 0 - Ошибка аргументов
- ARGS_OK 1 - Аргументы корректны
- ARGS_HELP 2 - Пользователь запросил справку

Изменение `parse_arguments()`

Расширение опций `getopt(argc, argv, "c:d:s:l:t:r:h")`

Добавились:

- t - лимит смены;
- r - `seed` генератора.

Добавлена обработка новых опций:

```
case 't':
    if (!parse_int_strict(optarg, &number) || number < 0)
    {
        fprintf(stderr, "Invalid shift limit '%s': minutes >= 0 are expected (0 = unlimited)\n", optarg);
        return ARGS_ERROR;
    }
    sim->shift_limit = number;
    break;

case 'r':
    if (!parse_int_strict(optarg, &number) || number < 0)
    {
        fprintf(stderr, "Invalid seed '%s': a non-negative integer is expected\n", optarg);
        return ARGS_ERROR;
    }
    sim->seed = (unsigned int)number;
    sim->random_calls = 1;
    break;
```

Добавлена проверка лишних аргументов

```c
if (optind < argc)
{
    fprintf(stderr, "Unexpected argument: %s\n", argv[optind]);
    print_usage(argv[0]);
    return ARGS_ERROR;
}
```

Изменение функции `open_log_file()`

```c
/**
 * @brief Opens the log file with the open(2) system call
 * @param sim Pointer to the simulation structure
 * @param path Path to the log file
 * @return 1 on success, 0 on failure
 */
static int open_log_file(Simulation *sim, const char *path)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);
    SOFT_ASSERT(path, "log path is NULL", 0);

    sim->log_fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (sim->log_fd < 0)
    {
        perror(path);
        return 0;
    }

    return 1;
}
```